Estándar de Comunicación Inalámbrica PC — ESP32 Maestro

Proyecto SOFÍA 2026

LabDST, ITESO

Entregable 2: Documento Formal del Estándar de Comunicación

Autores:

- Paola Monserrat Covarrubias Viveros
- Lizeth Anais Méndez Ortiz
- Juan Bernardo Orozco Quirarte
- Karen Itzel Sánchez García
- Alan Esaú Fuentes Gutiérrez

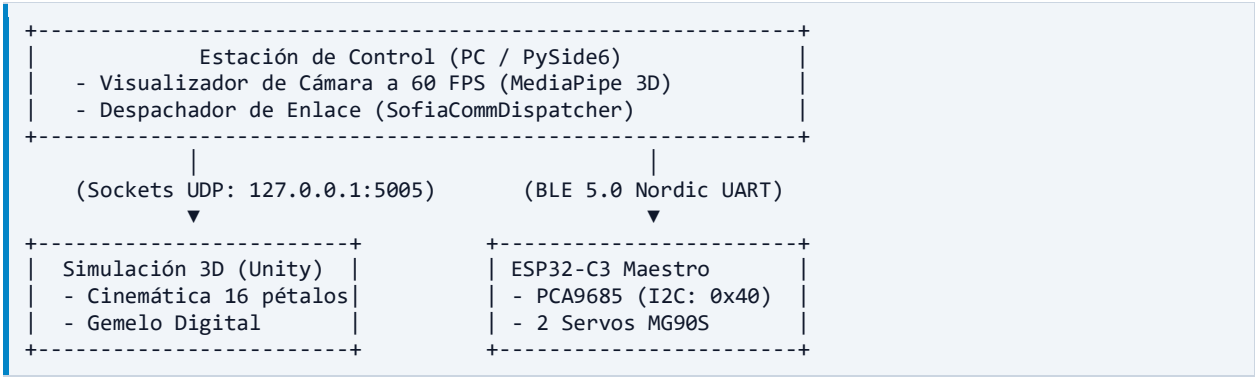
---

1. Introducción y Arquitectura del Enlace

El presente documento establece la especificación técnica formal del protocolo de comunicación entre la estación de control (PC) y el microcontrolador maestro (ESP32-C3) encargado de accionar los servomotores en los módulos cinemáticos de SOFÍA.

El protocolo cubre dos canales de transmisión:

1. **Canal Inalámbrico Primario (Bluetooth Low Energy - BLE):** Enlace hacia el hardware físico para posicionamiento articular y ejecución de rutinas coreográficas.
2. **Canal de Simulación (Sockets UDP):** Enlace hacia el gemelo digital en Unity / Webots para validación cinemática previa.



---

2. Parámetros de la Capa de Enlace

2.1 Canal de Simulación (UDP)

- **Protocolo:** UDP / IPv4.
- **Dirección IP de destino:** `127.0.0.1`.
- **Puerto:** `5005`.
- **Formato de datos:** Cadena ASCII con valores articulares delimitados por comas: `q1,q2`.

2.2 Canal Físico Inalámbrico (BLE Nordic UART Service)

El enlace opera bajo el perfil GATT serial estándar:

- **Nombre de difusión BLE:** `ESP32-Servos`.
- **Dirección MAC por defecto:** `70:AF:09:15:23:42`
- **UUID del Servicio UART:** `6e400001-b5a3-f393-e0a9-e50e24dcca9e`.
- **UUID Característica RX (PC → ESP32):** `6e400002-b5a3-f393-e0a9-e50e24dcca9e`.
- *Modo:* `Write` / `Write Without Response`.
- **UUID Característica TX (ESP32 → PC):** `6e400003-b5a3-f393-e0a9-e50e24dcca9e`.
- *Modo:* Notify. (Canal de retorno para emisión de ACKs y telemetría).

2.3 Asignación de Canales y Restricciones Mecánicas

Parámetros comprobados en el código fuente del firmware (`code/t firmware-mecatronica/arduino.ino`):

- **Dirección I2C del PCA9685:** `0x40`.
- **Canal Servo Q1 (Yaw horizontal):** Canal 14.
- **Canal Servo Q2 (Pitch vertical):** Canal 15.
- **Frecuencia PWM:** 50 Hz.
- **Rango angular seguro:** $60° \leq q \leq 120°$
- **Posición neutra (Home):** $q_1 = 90°$, $q_2 = 90°$
- **Posición de reposo:** $q_1 = 90°$, $q_2 = 60°$

---

3. Estructura de las Tramas de Comunicación

Se define una estructura delimitada por caracteres centinela para garantizar integridad y evitar errores por fragmentación en el búfer serial.

3.1 Trama de Solicitud (PC → ESP32)

```
$<SEQ>,<CMD>,<PARAM_1>,<PARAM_2>#
```

- `$` : Carácter de inicio de trama (Start of Frame).
- `<SEQ>` : Número de secuencia entero incremental (0 a 65535).
- `<CMD>` : Comando alfanumérico en mayúsculas.

- `<PARAM_N>` : Argumentos numéricos o de texto según el comando (opcionales).
- `#` : Carácter de fin de trama (End of Frame).

Ejemplos:

- Comando de ángulos coordinados: `$101,ANG,90,110#`
- Comando individual de eje horizontal: `$102,Q1,75#`
- Disparo de rutina coreográfica: `$103,ROUTINE,SALUDO#`
- Prueba de conectividad: `$104,PING#`

3.2 Trama de Confirmación / ACK (ESP32 → PC)

La respuesta del microcontrolador permite asociar el resultado de una operación con la solicitud que la originó. Se utilizará la siguiente estructura:

```
$<SEQ>,<STATUS>,<DATA>#
```

- `$` : Carácter de inicio de trama.
- `<SEQ>` : Número de secuencia del comando al que se responde.
- `<STATUS>` : `ACK` si el comando fue ejecutado correctamente; `NACK` si fue rechazado.
- `<DATA>` : Datos de confirmación o código de error.
- `#` : Carácter de fin de trama.

Ejemplos:

- Confirmación de ángulos ejecutados: `$101,ACK,OK,90,110#`
- Confirmación de rutina iniciada: `$103,ACK,STARTED,SALUDO#`
- Rechazo por ángulo fuera de rango: `$105,NACK,ERR_01#`

La estación de control deberá asociar cada respuesta con su número de secuencia correspondiente. Si no se recibe una respuesta dentro del tiempo de espera configurado, la PC podrá aplicar la política de reintentos definida en la Sección 6.2.

---

4. Identificación de Comandos del Sistema

Comando	Argumentos	Descripción	Respuesta del Sistema
`PING`	Ninguno	Prueba de vida del enlace inalámbrico.	`$SEQ,ACK,PONG#`
`ANG`	`q1,q2` (Enteros: 60 a 120)	Asigna ángulos simultáneos a q_1 y q_2.	`$SEQ,ACK,OK,q1,q2#`
`Q1`	`angulo` (Entero: 60 a	Asigna ángulo	`$SEQ,ACK,OK,q1#`

	120)	individual al eje horizontal.	
Q2	angulo (Entero: 60 a 120)	Asigna ángulo individual al eje vertical.	$SEQ,ACK,OK,q2#
ROUTINE	SALUDO | CORAZON	Ejecuta rutina coreográfica almacenada en el ESP32.	$SEQ,ACK,STARTED,<RUTINA>#
HOME	Ninguno	Centra ambos servomotores a posición neutra (90°, 90°).	$SEQ,ACK,OK,90,90#
REST	Ninguno	Coloca los actuadores en posición de reposo (90°, 60°).	$SEQ,ACK,OK,90,60#
STOP	Ninguno	Desactiva señales PWM en PCA9685 para liberar torque.	$SEQ,ACK,STOPPED#

5. Códigos de Error y Respuestas de Confirmación (ACK / NACK)

Cuando el microcontrolador detecta una anomalía en la trama recibida o una instrucción no permitida, devuelve un NACK con un código normalizado:

Código de Error	Nemónico	Causa del Fallo	Acción del Sistema en la GUI
ERR_01	OUT_OF_RANGE	Parámetro angular fuera del intervalo seguro (<60° o >120°).	La GUI satura la posición al límite físico más cercano y registra advertencia.
ERR_02	UNKNOWN_CMD	Comando no registrado en el catálogo oficial.	Se descarta el comando y se notifica en la terminal de eventos.
ERR_03	BAD_FORMAT	Trama sin delimitadores $ / # o con parámetros incompletos.	Se descarta el paquete y se registra error de sintaxis en el log.
ERR_04	ROUTINE_BUSY	Solicitud angular	Se omite el movimiento

		recibida mientras se ejecuta una rutina local activa.	para proteger la trayectoria coreográfica en curso.
`ERR_05`	`I2C_FAIL`	El driver PCA9685 no responde en la dirección I2C `0x40`.	Se emite alerta crítica de hardware en el panel de control.

---

6. Acknowledgements y Reglas de Comunicación

6.1 Flujo de Intercambio de Mensajes

```
     Estación PC (GUI)                              ESP32 Maestro
            │                                            │
            │── $201,ANG,90,105# ──────────────────────►│  (Valida formato y límites)
            │                                            │  (Aplica PWM en PCA9685)
            │◄── $201,ACK,OK,90,105# ──────────────────│  (Emite notificación TX Notify)
            │                                            │
  (Transacción completada)                               │
```

6.2 Parámetros de Temporización y Control de Flujo

- **Timeout de Respuesta (T_timeout):** 200 ms. Si tras 200 ms no se recibe la trama de confirmación, la PC considera el paquete no atendido.
- **Política de Reintentos:** Máximo 2 reintentos antes de reportar pérdida de comunicación con el robot físico.
- **Limitación de Tasa (Rate Limiting):** Durante el modo de seguimiento automático 3D, la tasa de transmisión se restringe a un máximo de 30 Hz (≈ 33.3 ms entre paquetes) para prevenir la saturación del búfer de la radio BLE del ESP32-C3.

---

7. Implementación de Referencia en Código

7.1 Módulo de Transmisión en Python (`code/communication/ble_sender.py`)

```
# Suscripción a la característica TX para recepción de confirmaciones
await client.start_notify(TX_CHARACTERISTIC_UUID, self._handle_ble_notification)

# Ensamblado de trama delimitada
trama = f"${self.seq_counter},{cmd},{params}#"
await client.write_gatt_char(RX_CHARACTERISTIC_UUID, trama.encode("utf-8"), response=False)
```

7.2 Implementación de la Característica TX en Firmware ESP32

Configuración requerida en el archivo de firmware (`code/tracking-vision/arduino.ino`):

```
#include <BLE2902.h>

// Puntero para la característica de transmisión TX
BLECharacteristic *pTxCharacteristic = nullptr;


#define TX_UUID "6e400003-b5a3-f393-e0a9-e50e24dcca9e"


// Configuración en setup():
pTxCharacteristic = service->createCharacteristic(
    TX_UUID,
    BLECharacteristic::PROPERTY_NOTIFY
);


// Emisión de confirmación ACK / NACK:
void responderACK(String seq, String status, String info) {
    if (pTxCharacteristic != nullptr) {
        String respuesta = "$" + seq + "," + status + "," + info + "#";
        pTxCharacteristic->setValue(respuesta.c_str());
        pTxCharacteristic->notify();
    }
}
```